Notas UCC: del código vulnerable a la mitigación

Integrantes
Doris Arzuaga
Jesús Egea
Diego Luna
Gabriela Zabaleta

Optativa
4 de noviembre de 2026

1. Contexto y arquitectura

1.1 Propósito, usuarios y tecnologías

"Notas UCC" es una aplicación web/API mínima que simula un sistema de consulta de calificaciones. El equipo la construyó para este proyecto con **Python 3, Flask 3 y SQLite**, y le introdujo a propósito dos fallos de naturaleza distinta que luego corrigió. Hay dos tipos de usuario: el **estudiante autenticado**, que solo debería ver sus propias notas, y el **visitante**, que solo puede intentar iniciar sesión.

1.2 Componentes y flujo de datos

Los componentes son cuatro. `POST /login` autentica y crea la sesión. `GET /notas/<id>` devuelve una calificación. La sesión es una cookie firmada con `secret_key` que guarda el `usuario_id`. La base SQLite tiene las tablas `usuarios` (credenciales) y `notas` (calificaciones, cada una con su `usuario_id`). El repositorio contiene las dos versiones: `app_vulnerable` (puerto 8000) y `app_corregida` (puerto 8001).

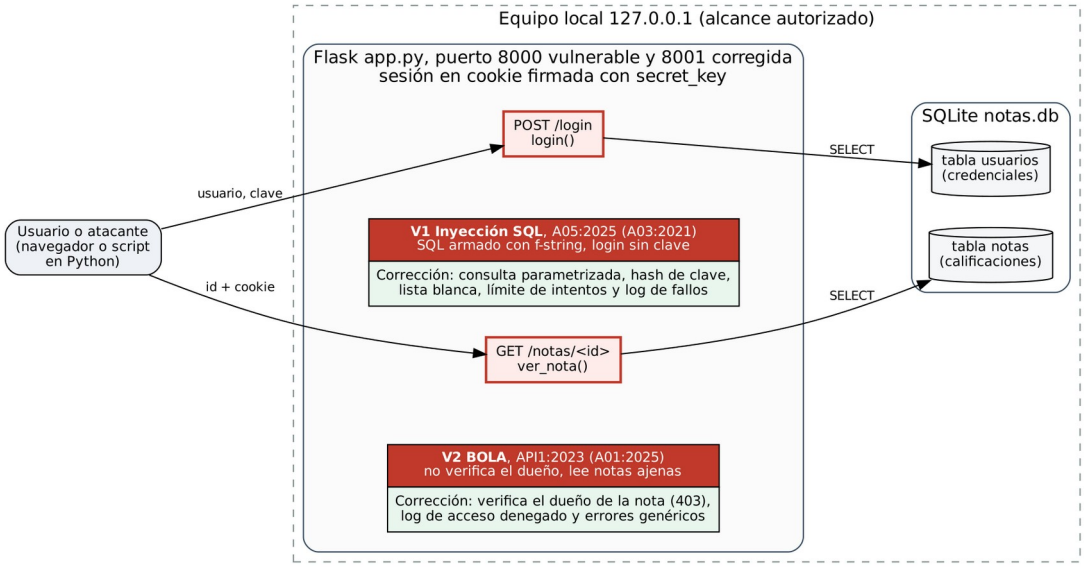


Figura 1. Arquitectura, flujo de datos y ubicación de cada vulnerabilidad y su control

1.3 Alcance autorizado de las pruebas

Las pruebas se limitaron a `127.0.0.1:8000` y `127.0.0.1:8001`, en los computadores de los integrantes y en el entorno aislado de integración continua. No se probó ningún sistema externo, institucional o de terceros. Las credenciales y los datos son ficticios.

2. Vulnerabilidades identificadas

2.1 Caracterización

Elemento	V1: Inyección SQL en `/login`	V2: BOLA en `/notas/<id>`
Descripción técnica	La consulta se arma concatenando con f-string el usuario y la clave recibidos, así que la entrada se interpreta como código SQL	La ruta exige sesión, pero devuelve la nota por su id sin comprobar que pertenezca a quien la pide

Elemento	V1: Inyección SQL en `/login`	V2: BOLA en `/notas/<id>`
Código afectado	`f"SELECT id, usuario FROM usuarios WHERE usuario = '{usuario}' AND clave = '{clave}'"`	`nota = db.execute("SELECT * FROM notas WHERE id = ?", (nota_id,))` seguido de `return jsonify(... dict(nota))`
Origen	**Implementación**: decisión de codificación insegura	**Diseño de la autorización**: falta la regla "solo el dueño ve su nota"
Propiedad comprometida	**Autenticidad** (suplantación); por extensión, confidencialidad e integridad de la cuenta	**Confidencialidad** de calificaciones ajenas
Afectados	Todos los usuarios (cualquier cuenta es suplantable); proceso de autenticación; tabla `usuarios`	Todos los estudiantes; tabla `notas`; cualquier endpoint futuro con el mismo patrón
Principio de desarrollo seguro	Separar código de datos; no confiar en la entrada; no guardar secretos en texto plano	Denegar por defecto; mínimo privilegio; autorización en cada petición y para cada objeto
Impacto potencial	Acceso total a cualquier cuenta sin conocer la clave, por parte de un visitante anónimo	Fuga masiva de notas: los ids son secuenciales y basta recorrerlos

2.2 Demostración controlada

Condición inicial: las dos apps recién iniciadas, que recrean `notas.db` con `ana` (notas 1 y 2) y `luis` (nota 3). *Herramienta:* script Python con `requests` (`pruebas/pruebas.py`), reproducible con un solo comando.

- **V1.** *Entrada:* `usuario=ana`, `clave=' OR '1'='1`. *Resultado:* 200, sesión abierta como ana. La condición queda `(usuario='ana' AND clave='') OR '1'='1'`: como `AND` tiene más precedencia que `OR`, es verdadera para todas las filas y se toma la primera. Una segunda entrada, `usuario=luis' --`, comenta el resto de la consulta y abre sesión **como luis**, de modo que el atacante elige a quién suplantar.
- **V2.** *Acción:* luis inicia sesión con su clave real y pide `GET /notas/2`. *Resultado:* 200 con la nota de ana (Optativa, 4.8). Basta cambiar un número en la URL.

3. Riesgo y relación con OWASP

3.1 Matriz de riesgo

El riesgo se calcula como probabilidad por impacto, cada uno de 1 a 3. Un puntaje de 1 o 2 es bajo, de 3 o 4 es medio, de 6 es alto y de 9 es crítico.

Vulnerabilidad	Probabilidad	Impacto	Nivel	Justificación
V1. SQLi	3: anónima, sin herramientas, payload público	3: suplantación de cualquier cuenta	**Crítico (9)**	Rompe la puerta de entrada; no requiere cuenta
V2. BOLA	3: cualquier cuenta legítima y un número en la URL	2: lectura (no modificación) de datos personales ajenos	**Alto (6)**	No depende de V1; patrón de diseño que se repetiría en cada endpoint

3.2 Relación con OWASP

V1 corresponde a OWASP Top 10 A05:2025 Injection (A03 en la edición 2021, CWE-89). Pertenece a esa categoría porque un dato no confiable termina interpretado como parte de un comando del intérprete SQL. Es exactamente la "consulta dinámica no parametrizada" que OWASP describe. La práctica insegura concreta es usar f-strings cuando la propia librería `sqlite3` ofrece parámetros ligados.

V2 corresponde a OWASP API Security Top 10 API1:2023 Broken Object Level Authorization (A01:2025 Broken Access Control, CWE-639). El control de acceso se queda en "¿hay sesión?" (autenticación) y nunca pregunta "¿esta sesión es dueña de *este* objeto?" (autorización a nivel de objeto). Además, el id que controla el usuario llega por la URL. Ese es el patrón exacto de BOLA.

3.3 Consecuencias y responsabilidad profesional

Para los usuarios: suplantación y exposición de calificaciones, con posible discriminación o presión académica. Para la organización: las notas son datos personales y la Ley 1581 de 2012 obliga a quien los trata a protegerlos con medidas técnicas (principio de seguridad), así que habría riesgo de sanción de la Superintendencia de Industria y Comercio. También habría pérdida de confianza y costos de respuesta al incidente. La responsabilidad recae en el equipo de desarrollo: ambos fallos se originan en decisiones propias, tienen controles estándar conocidos y no se habrían detectado sin pruebas de seguridad deliberadas.

4. Mitigaciones implementadas

4.1 V1: consulta parametrizada y clave con hash

```
fila = db.execute("SELECT id, usuario, clave_hash FROM usuarios WHERE usuario = ?",
(usuario,)).fetchone()
if fila and check_password_hash(fila["clave_hash"], clave):
```

1. *Causa que corrige:* el dato ya no puede alterar la estructura del SQL, porque viaja como parámetro y no como texto del comando. La clave tampoco forma parte de la consulta: se compara contra un hash.
2. *Principio:* separar código de datos, y no almacenar secretos en texto plano.
3. *Por qué se eligió:* es el control que recomienda OWASP como primera opción, corrige la causa (no un síntoma, como pasaría al filtrar comillas) y no cambia el comportamiento legítimo.
4. *Riesgo residual:* claves débiles y ausencia de MFA.

4.2 V2: verificación de propiedad del objeto

```
if nota["usuario_id"] != session["usuario_id"]:
    log_seguridad.warning("acceso_denegado usuario_id=%s nota_id=%s ...")
    return jsonify({"ok": False, "error": "No autorizado"}), 403
```

1. *Causa que corrige:* la autorización se evalúa en cada petición y para el objeto concreto, no solo por "hay sesión".
2. *Principio:* mínimo privilegio y denegar por defecto.
3. *Por qué se eligió:* es el control más directo para un BOLA y no exige cambiar el modelo de datos.
4. *Riesgo residual:* la regla vive dentro de cada endpoint y uno nuevo podría olvidarla. Además, el 403 confirma que el id existe (enumeración menor).

4.3 Defensa en profundidad

Se agregaron capas adicionales para que un fallo aislado no deje expuesto el sistema.

Control	Qué aporta
Validación server-side con lista blanca `[a-z0-9_]{1,30}` para el usuario	Rechaza entradas con comillas o comentarios antes de llegar a la base
Rate limiting: 5 fallos en 60 s, luego 429	Frena la fuerza bruta contra el login
Registro de eventos en `seguridad.log` (fallos, bloqueos, accesos denegados; nunca claves)	Detección y trazabilidad del abuso
`secret_key` desde la variable de entorno `NOTAS_SECRET_KEY` (aleatoria si falta)	Elimina el secreto fijo del código (CWE-259)
Cookie `HttpOnly` y `SameSite=Lax` (`Secure` en HTTPS); `session.clear()` al iniciar sesión	Reduce robo de sesión, CSRF y fijación de sesión
Cabeceras `nosniff`, `X-Frame-Options: DENY`, CSP, `Cache-Control: no-store`	Endurece la respuesta HTTP
Manejadores de error JSON genéricos (401/403/404/429/500)	No revelan trazas ni detalles internos

5. Resultados de validación

5.1 Matriz de validación

Se ejecutó la misma PoC (`pruebas/pruebas.py`) contra las dos versiones y una batería de 18 pruebas automatizadas con pytest (`pruebas/test_seguridad.py`), que también corre en GitHub Actions. Las salidas completas están en la carpeta `evidencias/` del repositorio.

Caso de prueba	Resultado vulnerable	Control aplicado	Resultado corregido
V1 `' OR '1'='1`	200, sesión sin clave	Parametrización y hash	401 genérico
V1 `luis' --`	200, sesión como luis	Lista blanca y parametrización	401 genérico
V2 luis lee la nota 2 (de ana)	200, nota ajena	Verificación de propiedad	403 y registro en log
Legítimo: nota propia y login real	200	Sin cambios	200
Errores: sin sesión e id inexistente	401 y 404	Errores genéricos JSON	401 y 404, sin detalles
7 claves erróneas seguidas	401 sin límite	Rate limiting	401 cinco veces y luego 429
Análisis estático (bandit)	B608 SQLi y B105 secreto fijo	Correcciones V1 y secretos	Sin hallazgos

5.2 Interpretación y riesgo residual

La PoC dio 16/16 casos con el resultado esperado y pytest 18/18. Los ataques que funcionaban en la versión vulnerable dejan de hacerlo en la corregida, la funcionalidad legítima es idéntica y los errores no filtran información interna. El **riesgo residual baja de Crítico (9) a Medio (3) en V1** (la probabilidad pasa a 1) y **de Alto (6) a Bajo (2) en V2**.

6. Reflexión ética y recomendaciones

6.1 Pregunta ética

¿Sería éticamente aceptable desplegar el sistema con la vulnerabilidad identificada para cumplir una fecha de entrega y corregirla posteriormente? No en este caso. *Protección de usuarios:* el daño recae en personas que no tomaron la decisión y que ni siquiera se enterarían, porque en la versión vulnerable la lectura de una nota ajena no deja ningún registro que la distinga de un acceso legítimo. *Gestión del riesgo:* los niveles son crítico y alto, la explotación es trivial y los controles cuestan pocas líneas de código, así que el beneficio de cumplir la fecha no compensa. *Transparencia:* desplegar sabiendo del fallo y sin informar ocultaría a los usuarios y a la organización un riesgo que les corresponde conocer. *Impacto organizacional:* hay obligación legal de proteger datos personales (Ley 1581 de 2012), además de riesgo reputacional. *Responsabilidad profesional:* el ingeniero responde por las consecuencias previsibles de lo que entrega. "Lo corregimos después" solo sería defendible con un riesgo bajo, documentado y aceptado formalmente por quien asume las consecuencias, con controles compensatorios y una fecha de corrección cierta. No es el caso de un fallo de autenticación o de control de acceso.

6.2 Práctica insegura frente a práctica segura

Construir SQL con strings frente a parámetros ligados. Comprobar solo "¿hay sesión?" frente a "¿es tuyo este objeto?". Una clave fija en el código frente a un secreto en el entorno. Mensajes de error que filtran detalles frente a respuestas genéricas con registro interno.

6.3 Seguridad en el ciclo de vida y en el despliegue

Fase	Práctica
Requisitos	Historias de abuso ("como estudiante, intento ver la nota de otro") y requisitos tomados de OWASP ASVS
Diseño	Modelado de amenazas del flujo de datos; matriz de autorización por rol y recurso; autorización centralizada (decorador o capa de servicio) en lugar de repetirla en cada endpoint
Implementación	Norma de equipo: solo consultas parametrizadas u ORM; revisión de código con lista de chequeo de seguridad en cada pull request
Pruebas	Pruebas de seguridad automatizadas (ataque y uso legítimo) como en `test_seguridad.py`; pruebas BOLA con dos sesiones en cada endpoint que devuelva datos de un usuario
Despliegue (CI/CD)	Pipeline `.github/workflows/seguridad.yml`: pytest, bandit (SAST) y pip-audit (dependencias); si algo falla, no se integra. Secretos solo en el gestor del entorno, HTTPS con `Secure`, servidor WSGI de producción (no el de desarrollo), permisos mínimos del pipeline
Operación	Monitoreo de `seguridad.log` con alertas por picos de 403/429; plan de respuesta a incidentes y revisión periódica de dependencias

6.4 Recomendaciones

Rate limiting persistente (por ejemplo, Redis), MFA, ids no secuenciales (UUID) y responder 404 ante objetos ajenos para evitar la enumeración.

7. Entregables y repositorio

Entregable	Dónde está
Informe ejecutivo	Este documento (`informe/`)
Código vulnerable y código corregido	`app_vulnerable/app.py` y `app_corregida/app.py`
Instrucciones de instalación y ejecución	`README.md`
Diagrama del sistema	Figura 1 y `diagrama.md`
Matrices de riesgo, validación y riesgo residual	Secciones 3 y 5, y `matrices.md`
Casos y scripts de prueba	`pruebas/pruebas.py` y `pruebas/test_seguridad.py`
Evidencias antes y después	Carpeta `evidencias/`
Integración continua	`.github/workflows/seguridad.yml`
Registro de participación	`PARTICIPACION.md`
Publicación técnica	README técnico en GitHub y `PUBLICACION.md`
Repositorio Git	https://github.com/Gabi2306/notas-ucc-seguridad, con el historial de commits del equipo

Todos los datos, usuarios y credenciales del proyecto son ficticios y las pruebas se ejecutaron solo en el entorno local autorizado.

8. Referencias

Congreso de la República de Colombia. (2012). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981

MITRE. (s. f.). *CWE-89: Improper neutralization of special elements used in an SQL command* y *CWE-639: Authorization bypass through user-controlled key*. https://cwe.mitre.org/

OWASP Cheat Sheet Series. (s. f.-a). *Authorization cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

OWASP Cheat Sheet Series. (s. f.-b). *Password storage cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

OWASP Cheat Sheet Series. (s. f.-c). *SQL injection prevention cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

OWASP Foundation. (2021). *A03:2021 – Injection*. En *OWASP Top 10:2021*. https://owasp.org/Top10/2021/A03_2021-Injection/

OWASP Foundation. (2023). *API1:2023 – Broken object level authorization*. En *OWASP API Security Top 10*. https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/

OWASP Foundation. (2025). *OWASP Top 10:2025*. https://owasp.org/Top10/

OWASP Foundation. (s. f.). *Application Security Verification Standard (ASVS)*. https://owasp.org/www-project-application-security-verification-standard/

Pallets Projects. (s. f.). *Security considerations*. En *Flask documentation*. https://flask.palletsprojects.com/en/stable/web-security/

PyCQA. (s. f.). *Bandit documentation*. https://bandit.readthedocs.io/